When SQLite starts, it gives you its version information and prompts you with `sqlite>`:

```
SQLite version 3.43.2 2023-10-10 13:08:14
Enter ".help" for usage hints.
sqlite>
```

Inside the SQLite shell, you can run SQL queries directly. To differentiate shell commands from SQL, SQLite uses a dot (.) prefix for its own commands. The welcome message tells you to use the `.help` command to find out more. The output of `.help` is rather lengthy—it is a full listing of everything you can do in the shell.

One of the commands available to you allows you to see all the tables in the database. Try the `.tables` command:

```
sqlite> .tables
```

Are there a few more tables than you expected? Remember, Django provides several built-in apps that you migrated, each of which resulted in a table. There are also system tables; `django_migrations` is where the ORM tracks which migrations have run. Your `Musician` class gets mapped in the `bands_musician` table:

```
auth_group                  bands_musician
auth_group_permissions      django_admin_log
auth_permission             django_content_type
auth_user                   django_migrations
auth_user_groups            django_session
auth_user_user_permissions
```

Django ORM Models name their tables *app*, underscore *(_)*, *classname*. For your `Musicians` class, that results in the *band* app name and the *musician* model name. This is only the default; you can control your table names if you wish (https://docs.djangoproject.com/en/5.0/ref/models/options/#db-table).

The `.schema` command shows you the SQL that was executed to create a table. Run this for the `bands_musician` table:

```
sqlite> .schema bands_musician
```

If you're new to SQL, the output can be a little daunting:

```
CREATE TABLE IF NOT EXISTS "bands_musician" ("id" integer NOT NULL
➥ PRIMARY KEY AUTOINCREMENT, "first_name" varchar(50) NOT NULL,
➥ "last_name" varchar(50) NOT NULL, "birth" date NOT NULL);
```

The advantage of an ORM is that you don't have to be an expert in SQL. Figure 4.4 breaks the statement down into its component parts.

The SQL that creates the table includes the `CREATE TABLE` command; the name of the table to be created (`bands_musician`); and four columns, `id`, `first_name`, `last_name`, and `birth`. Your `Musicians` model did not define an `id` attribute, as Django does that for you automatically. It uses an `integer` and declares that it can't be